EHMX の構文 型付け 制約生成と証明状況

EHMX は、抽象的な制約モデル X に基づく HM(X) に、代数的効果の操作、deep handler、値制限付き let を加えた体系である。構文・型付け・制約生成・意味論・効果付き進行性と、型・効果代入、スキーム値代入、β・let・return の保存を実装した。保存定理、推論の健全性・完全性・主要制約性、任意の到達状態の効果安全性は未証明である。

1 目標と出典

制約生成の健全性は ATTAPL 第10章の定理10.4.4、完全性は定理10.4.8、主要制約性は429–430頁の説明に対応する EHMX 拡張である。ATTAPL §10.5の型安全性とは別に、handler を含む評価に対する保存と効果付き進行性を証明する。既存 HMX の証明を、拡張構文にそのまま適用したとは主張しない。

操作・handler の参照は lean-system-f-xi のコミット 9173a02d9c67eea0b0c665d3319d834e3947518f、SystemFXi.Poly の EffectPolyTyping、EffectPolySemantics、EffectPolyPreservation、EffectPolySafety である。同リポジトリには let がないため、let の型付け・推論・簡約は EHMX で追加した。参照先の明示的な型抽象・型適用、高階の多相型は EHMX の項構文には含めない。

2 構文と束縛

項は変数、lambda 抽象、適用、let、perform、handle からなる。handler は return 節と一つの操作節を持つ。値は変数と lambda 抽象だけである。関数型は引数型、潜在効果、結果型の三つの成分を持つ。

型 kind は0、効果 kind は1である。型・効果は同じ生の構文 Ty で表し、Ty.HasKind で区別する。効果の構成子は空集合、操作ラベルの単集合、和集合であり、効果変数も許す。X は追加の kind、型構成子、述語を持てる。

項変数と型・効果変数には別の添字列を使う。自由変数は名前、束縛変数は de Bruijn 添字で表す。項の return 節では添字0が戻り値、操作節では添字0が継続 k、添字1が操作引数 x である。操作宣言の量化変数は種類のリストの先頭から添字0、1、…となる。

OpSignature.WF は、各宣言の量化変数が宣言された kind を持ち、引数型・結果型がその量化変数の下で kind 0を持ち、自由な型・効果変数を含まないことを要求する。操作宣言は型変数と効果変数を混在して量化できる。

3 制約モデル

Model は kind ごとの値、構成子の解釈、述語、部分順序、効果値の操作集合としての解釈を与える。EffectFramework は kind の非空性、構成子の kind 保存、部分順序の反射律・推移律・反対称律、有限集合効果の包含、空集合・単集合・和集合の解釈、関数型の順序を要求する。

関数型の順序は引数に反変、結果と潜在効果に共変である。保存の目標には、追加の非関数構成子が関数型の下に入らない ArrowSeparated を明記する。これらの前提に、型付け・推論・保存・進行の結論は含めない。

Concrete.framework は、基底型と構造的な効果付き関数型、有限集合効果からなる具体モデルについて、すべての EffectFramework の前提を証明する。Concrete.arrowSeparated も証明済みである。具体モデルの成立は、体系全体の保存定理の証明とは異なる。

制約は真・偽・原子・連言・存在量化・全称量化・スキームの束縛・変数のインスタンス化を持つ。全称制約は、指定 kind を満たすすべての値の組について成立することを要求する。型スキームの全称記号は、具体化できる型の上方閉集合を定めるものであり、全称制約とは意味が異なる。

4 型付け

Typing の引数は型シグネチャ S、モデル M、操作シグネチャ sig、型・効果変数の文脈 Delta、制約 C、項環境 Gamma、項 e、結果型 T、評価効果 eps である。JudgmentWF は制約・環境・型・効果の整形式性と項の作用域を要求する。外側の制約と環境のスキームは自由な項識別子に依存しない。

変数はスキームを具体化し、評価効果は空集合となる。lambda の本体の効果は関数型の潜在効果となり、lambda 自体の評価効果は空集合となる。適用の効果は関数式、引数式、潜在効果の和集合である。

letMono は右辺の型を単相で共有し、右辺と本体の効果を合併する。この規則は右辺が値の場合にも適用できる。letPoly は右辺が値の場合だけ適用し、右辺を空効果で検査する。新しく一般化する変数は外側の文脈の前に導入し、外側の制約と環境を shift する。一般化の制約に存在する具体化があることを含意として要求する。

perform は宣言の型・効果引数を具体化して引数を検査し、評価効果に操作ラベルを追加する。項に型・効果引数は保存しない。

handle は入力 A ! epsIn、出力 B ! epsOut に対し、epsIn が {op} と epsOut の和集合に含まれることを要求する。return 節は x:A の下で B ! epsOut を持つ。操作節は宣言の量化変数を任意のものとして、x:OpArg と k:(OpResult ->[epsOut] B) の下で検査する。外側の B、epsOut、制約、環境は量化変数の分だけ shift される。操作節の変数を外側へ漏らさない。

部分型・効果包含、制約の強化、型・効果変数の存在量化を型付け規則として持つ。存在量化の除去・反転では、内側で使う型を外側へ勝手に取り出してはならない。

5 評価

左から右の値呼び評価を行う。let は右辺だけを先に評価し、値になった時点で本体へ捕獲回避代入する。操作引数も値になるまで評価する。

handler は最も内側の対応する操作を捕捉する。捕捉する操作が途中の評価文脈で既に処理されていないことを、EvalCtx.handled により検査する。再開する継続は lambda y. handle E[y] with h であり、元の handler を再設置する deep handler である。操作節そのものを自動的に同じ handler の内側へ置くことはしない。

操作節への同時代入は [継続, 引数値] の順序で行う。継続の lambda の下へ移る際には、捕捉した文脈、return 節、操作節の外側の変数をそれぞれ正しい束縛深さで shift する。型・効果の引数は実行時には存在しない。

6 制約生成

generate sig e T eps は、要求型 T と評価効果上界 eps に対する制約を構文再帰で生成する。infer sig Gamma e T eps は環境のスキーム束縛とその実現可能性を付加する。型・効果の内部変数は束縛添字で導入するため、自由名の新鮮性を選ぶ処理は不要である。

値 let は右辺を空効果で生成し、その結果型をスキーム化する。非値 let は右辺と本体の両方の外側に型・効果変数を存在量化し、一つの単相型を共有する。handler は入力型・入力効果・内部出力型・内部出力効果を

存在量化し、操作節の制約だけを宣言の変数で全称量化する。最後に内部出力から要求型・要求効果への包含を課す。

未宣言の perform と handler の生成結果は偽制約になる。健全性の目標には Expr.Declared を明記した。これは計画の「整形式の入力」に必要な操作の作用域条件を具体化したものである。未宣言操作について偽制約の下で型付けを構成できるとは主張しない。unknownPerformNotTypable は、未宣言操作は任意の制約の下でも型付けできないことを証明する。

generationWF は生成制約の kind と束縛の整形式性を証明する。generationScope は、操作が宣言済みなら生成制約の自由項識別子が元の項の自由変数と一致することを証明する。これらは生成制約の充足と型付けの対応そのものではない。

7 証明済みの保証

progressGeneral は EffectFramework、型付け、空の項環境、kind を満たす割当て、および制約を実際に満たす割当てを前提とする。結論は、項が値であるか、一歩簡約できるか、解釈された評価効果に含まれる未処理操作を露出するかのいずれかである。

progressEffectSafety はこの結果を ProgressGoal に適用する。emptyEffectProgress は空効果と充足可能な制約の下で、現在の項が値または簡約可能であることを示す。到達したすべての状態の保証には保存定理が必要であり、まだ証明していない。

typingTypeEffectSubstitution は、型・効果代入を制約・スキーム・環境・結果型・効果に同時に適用して型付けを保つ。substBoundSemantics と entailsSubstBound は存在量化・全称量化を含む意味解釈と含意の保存を示す。型文脈のすべての kind が宣言済みという前提や、意味論の存在証人を型構文で表せるという前提は追加していない。

schemeValueSubstitution は SchemeValueSubstitutionGoal の正確な前提・結論を満たす。typingValueSubstitutionAux の帰納仮定は強い制約について一般化し、存在量化の内側で外側制約の shift と内側制約を合わせて代入する。RealizesScheme はこの帰納法で値の各具体化を扱う局所前提であり、generalizedValueRealizes で一般化導出から証明している。

betaPreservation、letPreservation、returnPreservation は、それぞれの簡約について型と効果の上界を保存する。let は単相・多相の両方を扱い、制約強化と存在量化を含む導出に適用できる。evalCtxReplacement は、すべての型文脈・制約・環境で成り立つ穴の型付け変換を、let 右辺を含む文脈全体へ持ち上げる。これらは閉項や具体モデルに限定した定理ではない。

operationClauseInstantiation は引数と継続の型を同じ型・効果引数で具体化するが、捕捉継続自体の型付けと操作節の二値同時代入は未証明である。この区別を保ち、操作捕捉を含む一歩保存、多歩保存、到達状態の効果安全性を完成とは扱わない。

その他の主要な証明済み補題は、operationInstantiation、kindShift、evalKind、evalShift、valuePure、closedValueLambda、evalCtxShiftPlug、schemeUpperSet、generationWF、generationScope である。各補題の使用箇所と残る証明義務は ehmx-todo.md に記録する。

8 未証明の最終定理と次の証明

Goals.lean の InferenceSoundnessGoal、InferenceCompletenessGoal、PrincipalityGoal、PreservationGoal、EffectSafetyGoal、EmptyEffectSafetyGoal は命題の定義であり、証明宣言ではない。完全性は環境実現可能性を要求する。主要制約性は要求型・効果を固定した含意の保証であり、ソルバーや主要解の存在ではない。

次は resumeTyping と OperationPreservationGoal を証明する。単相・多相の値代入は証明済みだが、捕捉継続の型付けを操作の同じ具体化に対応させる必要がある。handler では OperationPreservationGoal の型付け導出を、存在量化と部分型の規則も含めて反転する。操作の具体化引数を、それを束縛する存在量化の外へ取り出さず、その文脈の内側で継続の型付けを構成する。

ソルバー、参照・ストア、組込み定数の簡約、複数操作節、shallow handler は対象外である。既存 HMX の証明や参照先リポジトリを変更していない。

9 再現と検証

lake build は HMX、EHMX、回帰例を検査する。回帰例は実行・生成・型付けを区別する。handlerGenerationWF は生成制約の整形式性であり、handler 全体の推論健全性ではない。identityTyping と polymorphicLetTyping は、推論定理を仮定しない実際の型付け導出である。追加した回帰例では β・単相 let・多相 let・return の導出に各保存補題を適用し、exists_ / strengthen / sub の重なりと let 右辺の文脈簡約も検査する。mixedClauseTyping と mixedClauseInstantiated は混合引数の操作節の型付けと具体化を検査する。tests/EHMXGoals.lean は完成した ProgressGoal と SchemeValueSubstitutionGoal の型を検査する。

仕様PDFは tools/build_ehmx_spec.py でこの文書から生成する。付録の構文・モデル前提・型付け・簡約・生成規則・最終目標の Lean 宣言はソースから直接取り込む。PDF内のコード表示に限り、Unicode 記号を ASCII 表記へ置換する。ソースのハッシュとPDFのハッシュを manifest に記録し、--check で古いPDFを検出する。ハッシュの一致は説明の数学的正しさの証明ではないため、規則・前提・証明状況も読み合わせる。

付録 Lean の定義と証明済み定理の型

以下はコード表示のため記号を ASCII に置換したもの。forall は全称、exists は存在、-> は矢印、subset は包含、empty は空集合、Delta と Gamma は文脈、eps は効果を表す。Goal は命題の定義であり、証明済みを意味しない。

項と型

EHMX/Syntax.lean

```
inductive Ty where
  | free : TVar -> Ty
  | bound : Nat -> Ty
  | con : Symbol -> List Ty -> Ty
  deriving BEq, Repr


abbrev Effect := Ty
def Ty.arrow (A : Ty) (eps : Effect) (B : Ty) : Ty := .con .arrow [A, eps, B]
def Ty.empty : Effect := .con .empty []
def Ty.singleton (op : Op) : Effect := .con (.operation op) []
def Ty.union (eps eta : Effect) : Effect := .con .union [eps, eta]
abbrev Effect.empty := Ty.empty
abbrev Effect.singleton := Ty.singleton
abbrev Effect.union := Ty.union

inductive Ty.HasKind (S : Signature) (Delta : List Kind) : Ty -> Kind -> Prop where
  | free : X.1  in  S.kinds -> HasKind S Delta (.free X) X.1
  | bound : Delta[i]? = some k -> k  in  S.kinds -> HasKind S Delta (.bound i) k
  | con : f.Declared S ->
      List.Forall2 (HasKind S Delta) ts (f.parameters S) ->
      HasKind S Delta (.con f ts) (f.result S)
```

項の構文

EHMX/Syntax.lean

```
inductive PVar where
  | free : Nat -> PVar
  | bound : Nat -> PVar
  deriving DecidableEq, Repr

inductive Expr where
  | ident : PVar -> Expr
  | lam : Expr -> Expr
  | app : Expr -> Expr -> Expr
  | letE : Expr -> Expr -> Expr
  | perform : Op -> Expr -> Expr
  | handle : Expr -> Op -> Expr -> Expr -> Expr
  deriving DecidableEq, Repr
```

操作宣言と整形式性

EHMX/Syntax.lean

```
structure OpDecl where
  kinds : List Kind
  input : Ty
  output : Ty
  deriving BEq, Repr
```

```
abbrev OpSignature := Op -> Option OpDecl


def Expr.Declared (sig : OpSignature) : Expr -> Prop
  | .ident _ => True
  | .lam t => t.Declared sig
  | .app t u | .letE t u => t.Declared sig /\ u.Declared sig
  | .perform op t => sig op != none /\ t.Declared sig
  | .handle t op r c => sig op != none /\ t.Declared sig /\ r.Declared sig /\ c.Declared sig


def OpSignature.WF (S : Signature) (sig : OpSignature) : Prop :=
  forall  op d, sig op = some d ->
    (forall  k  in  d.kinds, k  in  S.kinds) /\ d.input.HasKind S d.kinds 0 /\
    d.output.HasKind S d.kinds 0 /\ d.input.fv = empty /\ d.output.fv = empty
```

制約とスキーム

EHMX/Syntax.lean

```
mutual
inductive Constraint where
  | truth | falsity
  | atom : Predicate -> List Ty -> Constraint
  | conj : Constraint -> Constraint -> Constraint
  | exists_ : List Kind -> Constraint -> Constraint
  | forall_ : List Kind -> Constraint -> Constraint
  | def_ : Scheme -> Constraint -> Constraint
  | defFree : Nat -> Scheme -> Constraint -> Constraint
  | inst : PVar -> Ty -> Constraint
  deriving Repr
inductive Scheme where
  | all : List Kind -> Constraint -> Ty -> Scheme
  deriving Repr
end
```

制約の意味

EHMX/Semantics.lean

```
mutual
def Constraint.Holds (M : Model) (rho : Assignment M) (bs : List M.Carrier)
    (iota : GroundEnv M) : Constraint -> Prop
  | .truth => True
  | .falsity => False
  | .atom p ts => p.eval M (ts.map (Ty.eval M rho bs))
  | .conj C D => C.Holds M rho bs iota /\ D.Holds M rho bs iota
  | .exists_ ks C => exists  vs, KindValues M ks vs /\ C.Holds M rho (vs ++ bs) iota
  | .forall_ ks C => forall  vs, KindValues M ks vs -> C.Holds M rho (vs ++ bs) iota
  | .def_ s C => C.Holds M rho bs (iota.push (s.denote M rho bs iota))
  | .defFree x s C => C.Holds M rho bs (iota.bind x (s.denote M rho bs iota))
  | .inst x T => T.eval M rho bs  in  iota.lookup x
def Scheme.denote (M : Model) (rho : Assignment M) (bs : List M.Carrier)
    (iota : GroundEnv M) : Scheme -> Set M.Carrier
  | .all ks C T => {v | M.kind v = 0 /\ exists  vs, KindValues M ks vs /\
      C.Holds M rho (vs ++ bs) iota /\ M.below 0 (T.eval M rho (vs ++ bs)) v}
end
```

モデルの全前提

EHMX/Semantics.lean

```
structure EffectFramework (S : Signature) (M : Model) : Prop where
```

```
  signature : S.WF
  inhabited : forall  k  in  S.kinds, exists  v, M.kind v = k
  constructorKind : forall  f, f.Declared S -> forall  vs, KindValues M (f.parameters S) vs ->
    M.kind (M.constructor f vs) = f.result S
  belowRefl : forall  k  in  S.kinds, forall  a, M.kind a = k -> M.below k a a
  belowTrans : forall  k  in  S.kinds, forall  a b c,
    M.kind a = k -> M.kind b = k -> M.kind c = k ->
    M.below k a b -> M.below k b c -> M.below k a c
  belowAntisymm : forall  k  in  S.kinds, forall  a b,
    M.kind a = k -> M.kind b = k -> M.below k a b -> M.below k b a -> a = b
  effectOrder : forall  a b, M.kind a = 1 -> M.kind b = 1 ->
    (M.below 1 a b <-> M.effects a  subset  M.effects b)
  effectsFinite : forall  a, M.kind a = 1 -> (M.effects a).Finite
  emptyEffect : M.effects (M.constructor .empty []) = empty
  singletonEffect : forall  op, M.effects (M.constructor (.operation op) []) = {op}
  unionEffect : forall  a b, M.kind a = 1 -> M.kind b = 1 ->
    M.effects (M.constructor .union [a,b]) = M.effects a  union  M.effects b
  arrowOrder : forall  a eps b a' eps' b',
    M.kind a = 0 -> M.kind eps = 1 -> M.kind b = 0 ->
    M.kind a' = 0 -> M.kind eps' = 1 -> M.kind b' = 0 ->
    (M.below 0 (M.constructor .arrow [a,eps,b]) (M.constructor .arrow [a',eps',b']) <->
      M.below 0 a' a /\ M.below 1 eps eps' /\ M.below 0 b b')


def ArrowSeparated (S : Signature) (M : Model) : Prop :=
  forall  f  in  S.constructors, S.result f = 0 -> forall  vs a eps b,
    KindValues M (S.parameters f) vs -> M.kind a = 0 -> M.kind eps = 1 -> M.kind b = 0 ->
    not  M.below 0 (M.constructor (.other f) vs) (M.constructor .arrow [a,eps,b])
```

型付けの全規則

EHMX/Typing.lean

```
def JudgmentWF (S : Signature) (Delta : List Kind) (C : Constraint)
    (Gamma : Env) (e : Expr) (T : Ty) (eps : Effect) : Prop :=
  C.WF S Delta 0 /\ C.fpi = empty /\ Gamma.WF S Delta /\ e.Scoped Gamma /\
  T.HasKind S Delta 0 /\ eps.HasKind S Delta 1

inductive Typing (S : Signature) (M : Model) (sig : OpSignature) :
    List Kind -> Constraint -> Env -> Expr -> Ty -> Effect -> Prop where
  | var : JudgmentWF S Delta C Gamma (.ident x) T .empty ->
      Gamma.lookup x = some s -> Entails S M Delta C (s.instanceConstraint T) ->
      Typing S M sig Delta C Gamma (.ident x) T .empty
  | lam : JudgmentWF S Delta C Gamma (.lam body) (.arrow A eps B) .empty ->
      Typing S M sig Delta C (Gamma.push (.mono A)) body B eps ->
      Typing S M sig Delta C Gamma (.lam body) (.arrow A eps B) .empty
  | app : JudgmentWF S Delta C Gamma (.app f x) B (.union (.union epsf epsx) eta) ->
      Typing S M sig Delta C Gamma f (.arrow A eta B) epsf ->
      Typing S M sig Delta C Gamma x A epsx ->
      Typing S M sig Delta C Gamma (.app f x) B (.union (.union epsf epsx) eta)
  | letMono : JudgmentWF S Delta C Gamma (.letE rhs body) B (.union epsrhs epsbody) ->
      Typing S M sig Delta C Gamma rhs A epsrhs ->
      Typing S M sig Delta C (Gamma.push (.mono A)) body B epsbody ->
      Typing S M sig Delta C Gamma (.letE rhs body) B (.union epsrhs epsbody)
  | letPoly : JudgmentWF S Delta C Gamma (.letE rhs body) B eps -> Value rhs ->
      Typing S M sig (ks ++ Delta) (.conj (C.shift ks.length 0) D)
        (Gamma.shift ks.length) rhs A .empty ->
      Entails S M Delta C (.exists_ ks D) ->
      Typing S M sig Delta C (Gamma.push (.all ks D A)) body B eps ->
      Typing S M sig Delta C Gamma (.letE rhs body) B eps
  | perform : JudgmentWF S Delta C Gamma (.perform op arg)
        (decl.output.openAt 0 args) (.union (.singleton op) eps) ->
      sig op = some decl -> List.Forall2 (Ty.HasKind S Delta) args decl.kinds ->
      Typing S M sig Delta C Gamma arg (decl.input.openAt 0 args) eps ->
      Typing S M sig Delta C Gamma (.perform op arg)
        (decl.output.openAt 0 args) (.union (.singleton op) eps)
  | handle : JudgmentWF S Delta C Gamma (.handle body op ret clause) B epsout ->
      sig op = some decl ->
```

```
      Typing S M sig Delta C Gamma body A epsin ->
      Entails S M Delta C (.le 1 epsin (.union (.singleton op) epsout)) ->
      Typing S M sig Delta C (Gamma.push (.mono A)) ret B epsout ->
      Typing S M sig (decl.kinds ++ Delta) (C.shift decl.kinds.length 0)
        (((Gamma.shift decl.kinds.length).push (.mono decl.input)).push
          (.mono (.arrow decl.output (epsout.shift decl.kinds.length 0)
            (B.shift decl.kinds.length 0))))
        clause (B.shift decl.kinds.length 0) (epsout.shift decl.kinds.length 0) ->
      Typing S M sig Delta C Gamma (.handle body op ret clause) B epsout
  | sub : JudgmentWF S Delta C Gamma e U eta -> Typing S M sig Delta C Gamma e T eps ->
      Entails S M Delta C (.le 0 T U) -> Entails S M Delta C (.le 1 eps eta) ->
      Typing S M sig Delta C Gamma e U eta
  | strengthen : JudgmentWF S Delta D Gamma e T eps -> Typing S M sig Delta C Gamma e T eps ->
      Entails S M Delta D C -> Typing S M sig Delta D Gamma e T eps
  | exists_ : JudgmentWF S Delta (.exists_ ks C) Gamma e T eps ->
      Typing S M sig (ks ++ Delta) C (Gamma.shift ks.length) e
        (T.shift ks.length 0) (eps.shift ks.length 0) ->
      Typing S M sig Delta (.exists_ ks C) Gamma e T eps
```

継続と簡約

EHMX/Operational.lean

```
def Expr.resume (E : EvalCtx) (op : Op) (ret clause : Expr) : Expr :=
  .lam (.handle ((E.shift 1 0).plug (.ident (.bound 0))) op
    (ret.shift 1 1) (clause.shift 1 2))

inductive Step : Expr -> Expr -> Prop where
  | beta : Value v -> Step (.app (.lam body) v) (body.openAt 0 [v])
  | letValue : Value v -> Step (.letE v body) (body.openAt 0 [v])
  | returnValue : Value v -> Step (.handle v op ret clause) (ret.openAt 0 [v])
  | operation {E : EvalCtx} : Value v -> op  notin  E.handled ->
      Step (.handle (E.plug (.perform op v)) op ret clause)
        (clause.openAt 0 [Expr.resume E op ret clause, v])
  | context {E : EvalCtx} : Step e e' -> Step (E.plug e) (E.plug e')

inductive Steps : Expr -> Expr -> Prop where
  | refl (e) : Steps e e
  | tail : Steps e e' -> Step e' e'' -> Steps e e''

def Unhandled (e : Expr) (op : Op) : Prop :=
  exists  (E : EvalCtx) (v : Expr), e = E.plug (.perform op v) /\ Value v /\ op  notin  E.handled
```

制約生成

EHMX/Inference.lean

```
def resultConstraint (A T : Ty) (eta eps : Effect) : Constraint :=
  .conj (.le 0 A T) (.le 1 eta eps)

def generate (sig : OpSignature) : Expr -> Ty -> Effect -> Constraint
  | .ident x, T, eps => .conj (.inst x T) (.le 1 .empty eps)
  | .lam body, T, eps =>
      .exists_ [0,1,0] (.conj
        (.def_ (.mono (.bound 0)) (generate sig body (.bound 2) (.bound 1)))
        (resultConstraint (.arrow (.bound 0) (.bound 1) (.bound 2))
          (T.shift 3 0) .empty (eps.shift 3 0)))
  | .app f x, T, eps =>
      .exists_ [0,1,1,1] (.conj
        (generate sig f (.arrow (.bound 0) (.bound 1) (T.shift 4 0)) (.bound 2))
        (.conj (generate sig x (.bound 0) (.bound 3))
          (.le 1 (.union (.union (.bound 1) (.bound 2)) (.bound 3)) (eps.shift 4 0))))
  | .letE rhs body, T, eps =>
      if rhs.isValue then
        .letBound (.all [0] (generate sig rhs (.bound 0) .empty) (.bound 0))
```

```
          (generate sig body T eps)
      else
        .exists_ [0,1,1] (.conj (generate sig rhs (.bound 0) (.bound 1))
          (.conj (.def_ (.mono (.bound 0)) (generate sig body (T.shift 3 0) (.bound 2)))
            (.le 1 (.union (.bound 1) (.bound 2)) (eps.shift 3 0))))
  | .perform op arg, T, eps =>
      match sig op with
      | none => .falsity
      | some decl =>
        .exists_ (decl.kinds ++ [1]) (.conj
          (generate sig arg decl.input (.bound decl.kinds.length))
          (resultConstraint decl.output (T.shift (decl.kinds.length + 1) 0)
            (.union (.singleton op) (.bound decl.kinds.length))
            (eps.shift (decl.kinds.length + 1) 0)))
  | .handle body op ret clause, T, eps =>
      match sig op with
      | none => .falsity
      | some decl =>
        let B := Ty.bound 2
        let eta := Ty.bound 3
        let n := decl.kinds.length
        .exists_ [0,1,0,1] (.conj (generate sig body (.bound 0) (.bound 1))
          (.conj (.le 1 (.bound 1) (.union (.singleton op) eta))
          (.conj (.def_ (.mono (.bound 0)) (generate sig ret B eta))
          (.conj (.forall_ decl.kinds
            (.def_ (.mono decl.input)
              (.def_ (.mono (.arrow decl.output (eta.shift n 0) (B.shift n 0)))
                (generate sig clause (B.shift n 0) (eta.shift n 0)))))
            (resultConstraint B (T.shift 4 0) eta (eps.shift 4 0))))))

def infer (sig : OpSignature) (Gamma : Env) (e : Expr) (T : Ty) (eps : Effect) : Constraint :=
  Gamma.letIn (generate sig e T eps)
```

証明済み：型・効果代入

EHMX/Proofs/TypingTypeSubstitution.lean

```
theorem typingTypeEffectSubstitution (hF : EffectFramework S M)
    (hsig : OpSignature.WF S sig) (ht : Typing S M sig Delta C Gamma e T eps) :
    forall  Delta' sigma, BoundSubst.Valid S Delta Delta' sigma ->
      Typing S M sig Delta' (C.substBound sigma) (Gamma.substBound sigma) e
        (T.substBound sigma) (eps.substBound sigma)
```

証明済み：スキーム値代入

EHMX/Proofs/ValueSubstitution.lean

```
theorem schemeValueSubstitution (S : Signature) (M : Model) (sig : OpSignature) :
    SchemeValueSubstitutionGoal S M sig
```

最終目標の全前提と結論

EHMX/Goals.lean

```
def Assumptions (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  EffectFramework S M /\ OpSignature.WF S sig


def InferenceSoundnessGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  Assumptions S M sig -> forall  Delta Gamma e T eps,
    (forall  k  in  Delta, k  in  S.kinds) -> Gamma.WF S Delta -> e.Scoped Gamma -> e.Declared sig ->
    T.HasKind S Delta 0 -> eps.HasKind S Delta 1 ->
    Typing S M sig Delta (infer sig Gamma e T eps) Gamma e T eps
```

```
def InferenceCompletenessGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  Assumptions S M sig -> forall  Delta C Gamma e T eps,
    (forall  k  in  Delta, k  in  S.kinds) -> Entails S M Delta C Gamma.exists_ ->
    Typing S M sig Delta C Gamma e T eps -> Entails S M Delta C (infer sig Gamma e T eps)

def PrincipalConstraint (S : Signature) (M : Model) (sig : OpSignature)
    (Delta : List Kind) (Gamma : Env) (e : Expr) (T : Ty) (eps : Effect) (D : Constraint) : Prop :=
  Typing S M sig Delta D Gamma e T eps /\
  forall  C, Entails S M Delta C Gamma.exists_ -> Typing S M sig Delta C Gamma e T eps -> Entails S M
    Delta C D

def PrincipalityGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  Assumptions S M sig -> forall  Delta Gamma e T eps,
    (forall  k  in  Delta, k  in  S.kinds) -> Gamma.WF S Delta -> e.Scoped Gamma -> e.Declared sig ->
    T.HasKind S Delta 0 -> eps.HasKind S Delta 1 ->
    PrincipalConstraint S M sig Delta Gamma e T eps (infer sig Gamma e T eps)

def SafetyAssumptions (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  Assumptions S M sig /\ ArrowSeparated S M

def PreservationGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  SafetyAssumptions S M sig -> forall  Delta C Gamma e e' T eps,
    Typing S M sig Delta C Gamma e T eps -> Step e e' -> Typing S M sig Delta C Gamma e' T eps


def ProgressGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  SafetyAssumptions S M sig -> forall  C e T eps,
    Typing S M sig [] C {} e T eps ->
    forall  rho iota, Assignment.Valid S M rho -> iota.Valid -> C.Holds M rho [] iota ->
      Value e \/ (exists  e', Step e e') \/ exists  op, Unhandled e op /\ op  in  M.effects (eps.eval M
    rho [])

def EffectSafetyGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  SafetyAssumptions S M sig -> forall  C e e' T eps,
    Typing S M sig [] C {} e T eps -> Steps e e' ->
    forall  rho iota, Assignment.Valid S M rho -> iota.Valid -> C.Holds M rho [] iota ->
      Value e' \/ (exists  e'', Step e' e'') \/
        exists  op, Unhandled e' op /\ op  in  M.effects (eps.eval M rho [])

def EmptyEffectSafetyGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  SafetyAssumptions S M sig -> forall  C e e' T,
    Satisfiable S M [] C -> Typing S M sig [] C {} e T .empty -> Steps e e' ->
      Value e' \/ exists  e'', Step e' e''


def SchemeValueSubstitutionGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  Assumptions S M sig -> forall  Delta C Gamma v body ks D A T eps,
    Value v -> Typing S M sig (ks ++ Delta) (.conj (C.shift ks.length 0) D)
      (Gamma.shift ks.length) v A .empty ->
    Entails S M Delta C (.exists_ ks D) ->
    Typing S M sig Delta C (Gamma.push (.all ks D A)) body T eps ->
    Typing S M sig Delta C Gamma (body.openAt 0 [v]) T eps


def OperationPreservationGoal (S : Signature) (M : Model) (sig : OpSignature) : Prop :=
  SafetyAssumptions S M sig -> forall  Delta C Gamma (E : EvalCtx) op ret clause v B eps,
    Value v -> op  notin  E.handled ->
    Typing S M sig Delta C Gamma (.handle (E.plug (.perform op v)) op ret clause) B eps ->
    Typing S M sig Delta C Gamma
      (clause.openAt 0 [Expr.resume E op ret clause, v]) B eps
```